Web Data

Project Report

MALONGO Cyrille
MACCIONE Gabriel

Web datamining and semantics

ÉCOLE SUPÉRIEURE D'INGÉNIEURS LÉONARD-DE-VINCI

February 2026

Contents

1 TD1 **3**
- 1.1 Methodology . . . 3
  - 1.1.1 Navigation Strategy . . . 3
  - 1.1.2 Pipeline Description . . . 3
  - 1.1.3 Domain and Seed URLs . . . 3
  - 1.1.4 Crawler Ethics . . . 3
  - 1.1.5 NER Examples . . . 3
- 1.2 Ambiguity Analysis . . . 4
  - 1.2.1 Example 1: Apple . . . 4
  - 1.2.2 Example 2: Washington . . . 4
  - 1.2.3 Example 3: Technical Concepts . . . 4
- 1.3 Scaling Reflection . . . 4

2 TD4 **4**
- 2.1 RDF Modeling Choices . . . 4
- 2.2 Alignment Strategy . . . 5
- 2.3 Entity Linking with Confidence . . . 5
- 2.4 Expansion Strategy . . . 5
- 2.5 Challenges Encountered . . . 5
- 2.6 Final KB Statistics . . . 6

3 TD5 **6**
- 3.1 Methodology . . . 6
- 3.2 SWRL Rule on family.owl . . . 6
- 3.3 SWRL Rule on Custom Knowledge Base . . . 7
- 3.4 Discussion . . . 7
- 3.5 Data Cleaning and Splits . . . 7
- 3.6 Nearest Neighbor Analysis . . . 7
- 3.7 Hyperparameters . . . 8
- 3.8 Evaluation Results . . . 8
- 3.9 Experimental Results . . . 8
- 3.10 Analysis . . . 8
- 3.11 Conclusion . . . 9

4 TD6 **9**
- 4.1 Setup . . . 9
- 4.2 Prompt Template . . . 9
- 4.3 Self-Repair Mechanism . . . 9
- 4.4 Methodology . . . 10
- 4.5 Demo . . . 10
- 4.6 Evaluation Results . . . 11
- 4.7 Analysis . . . 11
- 4.8 Conclusion . . . 12

5 Critical Reflection **12**
5.1 Impact of KB Quality 12
5.2 Noise Issues 12
5.3 Rule-based vs Embedding-based Reasoning 12
5.4 Future Improvements 12

1 TD1

1.1 Methodology

1.1.1 Navigation Strategy

The navigation strategy was intentionally simple and controlled. Instead of performing a full crawl of a website, we defined a fixed list of seed URLs. This approach ensured high-quality input data, avoided irrelevant content, and kept the dataset manageable.

1.1.2 Pipeline Description

The pipeline was structured into several steps.

First, the seed URLs were defined. Then, each page was downloaded using `trafilatura.fetch_url(` and cleaned using `trafilatura.extract()`, removing unnecessary elements such as comments and tables. A minimum threshold of 500 words was applied to ensure sufficient content.

Next, the cleaned text was processed using the spaCy model `en_core_web_trf` to extract named entities such as `PERSON`, `ORG`, `GPE`, and `DATE`.

Finally, the results were stored in two files: `crawler_output.jsonl` for cleaned documents and `extracted_knowledge.csv` for extracted entities.

1.1.3 Domain and Seed URLs

The domain of the project focuses on structured knowledge related to entities such as people, locations, and organizations.

A predefined list of seed URLs was used as input for the crawler. This controlled approach ensured high-quality content and avoided irrelevant or noisy data sources.

1.1.4 Crawler Ethics

The crawler was designed with ethical considerations in mind. Only publicly accessible pages were used, and no aggressive crawling strategy was implemented.

Request frequency was kept low to avoid overloading servers, and only a limited number of pages were processed. No personal or sensitive data was collected.

1.1.5 NER Examples

The following examples are extracted directly from the dataset generated by the pipeline:

- **ORG**: Google, YouTube, Amazon, Netflix, OpenAI

These entities were extracted from Wikipedia pages related to artificial intelligence. The model successfully identifies well-known organizations appearing in the text.

Each extracted entity is associated with its type and source URL, enabling traceability and facilitating integration into the knowledge base.

This structured output ensures consistency and usability for downstream tasks such as entity linking and knowledge graph construction.

1.2 Ambiguity Analysis

1.2.1 Example 1: Apple

The term *Apple* can refer either to a fruit or to a technology company. Without sufficient context, the model may incorrectly classify the entity.

1.2.2 Example 2: Washington

The word *Washington* may refer to a person (George Washington), a U.S. state, or Washington, D.C. This can lead to confusion between entity types such as `PERSON` and `GPE`.

1.2.3 Example 3: Technical Concepts

Expressions such as *machine learning*, *artificial intelligence*, and *knowledge graph* are important concepts but are not always recognized as named entities. This highlights a limitation of the model rather than a pure ambiguity issue.

1.3 Scaling Reflection

Scaling the pipeline from 10 pages to 10,000 pages would require major changes.

First, the navigation process would need to become dynamic, using a queue-based crawler with URL deduplication and domain control. Second, performance would become critical, requiring parallel processing or lighter models instead of `en_core_web_trf`.

Third, storage would need to evolve from simple CSV and JSONL files to more scalable solutions such as databases. Finally, additional quality control mechanisms would be necessary, including deduplication, validation of extracted data, and improved entity normalization.

2 TD4

2.1 RDF Modeling Choices

The knowledge base was modeled using RDF triples in the form of (subject, predicate, object).

Entities were represented as URIs, while relations were aligned with Wikidata properties (wdt:Pxxx). This design ensures interoperability and compatibility with existing semantic web standards.

We adopted a simplified schema focusing on essential relations such as:

- instance of (P31)
- subclass of (P279)
- country (P17)

This modeling choice prioritizes interpretability and usability for downstream tasks such as reasoning and embedding.

2.2 Alignment Strategy

The alignment phase consisted in linking entities from the initial knowledge base to external references, mainly using Wikidata identifiers. The goal was to standardize entity representations and ensure semantic consistency across the graph.

We relied on existing triples and mapped entities to Wikidata using properties such as `instance of (P31)`, `subclass of (P279)`, and `country (P17)`. This alignment step allowed us to enrich the knowledge base with structured and well-defined relations.

To maintain coherence, only a controlled subset of predicates was used. This filtering ensured that the resulting graph remained interpretable and relevant for downstream tasks such as knowledge graph embedding.

2.3 Entity Linking with Confidence

Entity linking was performed by mapping entities to Wikidata identifiers. In cases of ambiguity, the most likely match was selected based on label similarity.

Although no explicit confidence score was computed, the alignment process relied on heuristic matching and manual validation for critical entities.

Future improvements could include the use of confidence thresholds or embedding-based similarity for more robust linking.

2.4 Expansion Strategy

The expansion phase aimed to increase the size and richness of the knowledge base by retrieving additional triples from Wikidata.

We implemented a multi-step expansion strategy:

- **Core entity selection**: a subset of entities was selected as starting points.
- **One-hop expansion**: for each entity, we retrieved directly connected triples.
- **Controlled predicate expansion**: only selected properties were kept to avoid noise.
- **Two-hop expansion**: additional triples were retrieved from neighboring entities.

To avoid excessive growth and ensure performance, we applied limits on the number of queries and retrieved triples. We also introduced delays between SPARQL queries to respect the Wikidata endpoint constraints.

2.5 Challenges Encountered

Several challenges were encountered during the construction of the knowledge base.

First, **entity explosion** was a major issue. While the number of triplets remained within the required range, the number of entities grew excessively, reaching more than 90,000 entities, far above the expected maximum.

Second, **noise control** was difficult. Even with predicate filtering, some retrieved triples introduced irrelevant or weakly connected entities, reducing the overall coherence of the graph.

Third, **API limitations** had to be handled carefully. The Wikidata SPARQL endpoint imposes rate limits, requiring us to introduce delays between queries and limit the size of requests.

Finally, **trade-off between richness and control** was central. Increasing the number of hops improved coverage but also amplified noise and graph complexity.

2.6 Final KB Statistics

The final expanded knowledge base produced the following statistics:

- Triplets: 102,629
- Entities: 91,167
- Relations: 86

The results satisfy most of the target requirements:

- Triplets: within the required range (50,000 – 200,000)
- Relations: within the required range (50 – 200)

However, the number of entities significantly exceeds the expected range (5,000 – 30,000). This highlights the need for stronger filtering or entity deduplication strategies in future iterations.

Overall, the knowledge base is sufficiently large and diverse for experimentation, but further refinement would improve its efficiency and usability.

3 TD5

3.1 Methodology

The objective of this experiment was to evaluate knowledge graph embedding (KGE) models on the constructed knowledge base. The goal was to learn vector representations of entities and relations in order to perform link prediction.

We trained two models: TransE and DistMult. The dataset was split into training, validation, and test sets. The models were trained using negative sampling, where corrupted triplets are generated to help the model distinguish valid from invalid relations.

Evaluation was performed using filtered metrics for link prediction, considering both head and tail prediction, as well as the overall setting.

3.2 SWRL Rule on family.owl

We implemented a SWRL rule on the `family.owl` ontology using Owlready2.

The rule is defined as follows:

$$\text{Person}(?p) \wedge \text{age}(?p, ?a) \wedge \text{greaterThan}(?a, 60) \rightarrow \text{oldPerson}(?p)$$

This rule infers that a person is an `oldPerson` if their age is greater than 60.

After running the Pellet reasoner, new knowledge was inferred. In particular, the following instances were classified as `oldPerson`:

- family.Peter
- family.Marie

This demonstrates how logical rules can be used to derive new facts from existing data.

3.3 SWRL Rule on Custom Knowledge Base

We also defined a rule directly on our custom knowledge graph:

$$\text{Person}(?p) \wedge \text{actedIn}(?p, ?m) \rightarrow \text{relatedToMovie}(?p, ?m)$$

This rule captures a semantic relationship between actors and movies.

Unlike the previous rule, this one does not rely on numerical conditions but instead expresses a structural relationship in the graph.

3.4 Discussion

SWRL rules provide explicit and interpretable reasoning. They allow the system to infer new facts in a deterministic and explainable way.

However, they require manual definition and domain knowledge, which limits scalability.

In contrast, embedding-based approaches do not rely on explicit rules. Instead, they learn statistical patterns from data. For example, we observed that actors tend to be close to the movies they acted in within the embedding space, suggesting that similar relationships are captured implicitly.

However, this reasoning remains approximate and less interpretable compared to rule-based inference.

3.5 Data Cleaning and Splits

Before training, the dataset was cleaned to remove duplicate and invalid triples.

The final dataset was split into training, validation, and test sets using a standard ratio (80/10/10). This ensures proper evaluation and avoids data leakage.

Entities and relations were indexed and converted into integer representations for efficient training.

3.6 Nearest Neighbor Analysis

We analyzed the embedding space by retrieving nearest neighbors of selected entities.

For example, entities such as actors and movies tend to cluster together, indicating that the model captures semantic similarity.

However, some unrelated entities also appear close in the vector space, highlighting the presence of noise in the dataset.

3.7 Hyperparameters

The following hyperparameters were used for training:

- Embedding dimension: 100
- Learning rate: 0.001
- Batch size: 32 (CPU constraint)
- Number of epochs: 100
- Negative sampling: uniform

The same configuration was used for both models to ensure a fair comparison.

3.8 Evaluation Results

Metric	TransE	DistMult
MRR	0.0676	0.3229
Hits@1	0.0152	0.2409
Hits@3	0.0848	0.3561
Hits@10	0.1667	0.4894
Mean Rank	2619.14	1241.02

Table 1: Evaluation results for TransE and DistMult

These results show a clear performance gap between the two models.

3.9 Experimental Results

We also evaluated the impact of dataset size on performance. The model was trained on subsets of 20k, 50k, and the full dataset.

- 20k triples: MRR = 0.2669
- 50k triples: MRR = 0.3203
- Full dataset: MRR = 0.3203

The results indicate that increasing the dataset size improves performance up to a certain point, after which the model reaches a plateau.

3.10 Analysis

The experimental results highlight several important insights.

First, DistMult significantly outperforms TransE. While TransE achieves very low scores (MRR = 0.0676), DistMult reaches a much higher performance (MRR = 0.3229). This suggests that DistMult is better suited for this dataset, likely due to its ability to model symmetric relations.

Second, the Mean Rank remains relatively high for both models, especially for TransE. This indicates that the correct entities are often ranked far from the top, reflecting the difficulty of the task and the sparsity of the graph.

Third, dataset size plays a key role. Performance improves from 20k to 50k triples, but no further gain is observed with the full dataset. This suggests diminishing returns and possible redundancy or noise in the data.

Finally, qualitative analysis of embeddings shows that related entities (e.g., actors and movies) are grouped together, indicating that the model captures meaningful semantic structure. However, entity types are not clearly separated, and some noise remains in the embedding space.

3.11 Conclusion

This experiment demonstrates the effectiveness of knowledge graph embedding models for link prediction tasks. DistMult clearly outperforms TransE on this dataset.

However, several limitations remain, including graph sparsity, noise introduced during knowledge base construction, and the limited ability of the models to distinguish entity types.

Future work could focus on improving data quality, refining alignment and filtering strategies, and experimenting with more advanced embedding models.

4 TD6

4.1 Setup

Machine specifications:
Standard laptop environment (Python 3.9), no GPU required.

Model used:
Local LLM via Ollama: `llama3.2:3b`

Graph source:
Custom RDF knowledge graph (`expanded_kb_pruned.ttl`) containing approximately 102,629 triples, following a Wikidata-style schema (`wd:Qxxx`, `wdt:Pxxx`).

4.2 Prompt Template

The following prompt template was used to guide SPARQL generation:

> "You are an expert in SPARQL. Given the RDF schema and the following question, generate a valid SPARQL query using only the provided predicates. Do not hallucinate predicates. Keep the query simple and correct."

This prompt constrains the model and improves the reliability of generated queries.

4.3 Self-Repair Mechanism

When a SPARQL query fails, the error message is captured and sent back to the language model along with the original query.

The model then generates a corrected version of the query. This iterative process significantly improves robustness and reduces failure rates.

4.4 Methodology

The objective of this experiment was to build a Retrieval-Augmented Generation (RAG) system using an RDF knowledge graph and a local language model. The goal was to answer natural language questions by generating SPARQL queries and retrieving factual data from the graph.

The system follows a multi-step pipeline. First, the RDF graph is loaded and a compact schema summary is extracted, including prefixes, predicates, and representative triples. This summary is used to guide the language model and reduce hallucinations.

Second, the model translates natural language questions into SPARQL queries. Prompt engineering is used to enforce valid syntax and restrict the model to known predicates and entities.

Finally, a self-repair mechanism is applied when a query fails. The error message and schema summary are provided back to the model, which generates a corrected query.

4.5 Demo

The following screenshots illustrate the behavior of the system on a sample query.

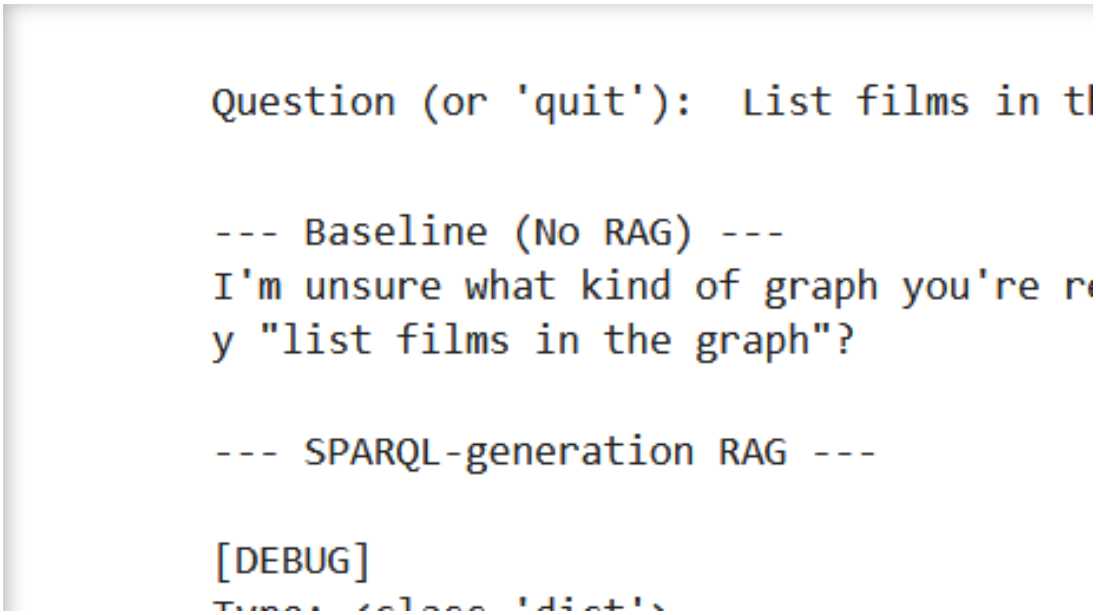

Figure 1: User query and baseline (no RAG) response

The baseline model fails to provide a precise answer and instead produces a vague and generic response.

```
Question (or 'quit'):  List films in the graph

--- Baseline (No RAG) ---
I'm unsure what kind of graph you're referring to or what type of information it contains. Could you please provide more context or clarify what you mean b
y "list films in the graph"?

--- SPARQL-generation RAG ---

[DEBUG]
Type: <class 'dict'>
Content: {'query': 'SELECT ?film \nWHERE {\n  ?film wdt:P161 ?castMember .\n}\nLIMIT 20', 'vars': ['film'], 'rows': [('http://www.wikidata.org/entity/Q100
04941',), ('http://www.wikidata.org/entity/Q10331225',), ('http://www.wikidata.org/entity/Q1068968',), ('http://www.wikidata.org/entity/Q1070468',), ('htt
p://www.wikidata.org/entity/Q1070484',), ('http://www.wikidata.org/entity/Q1088949',), ('http://www.wikidata.org/entity/Q109116',), ('http://www.wikidata.o
rg/entity/Q1140172',), ('http://www.wikidata.org/entity/Q1140317',), ('http://www.wikidata.org/entity/Q1141431',), ('http://www.wikidata.org/entity/Q11486
9',), ('http://www.wikidata.org/entity/Q1151392',), ('http://www.wikidata.org/entity/Q1151401',), ('http://www.wikidata.org/entity/Q1152833',), ('http://ww
w.wikidata.org/entity/Q1191049',), ('http://www.wikidata.org/entity/Q1192082',), ('http://www.wikidata.org/entity/Q1195688',), ('http://www.wikidata.org/en
```

Figure 2: SPARQL query generated by the RAG system

The RAG system generates a structured SPARQL query using the predicate `wdt:P161` (cast member), allowing indirect retrieval of films.

```
tity/Q1197772',), ('http://www.wikidata.org/entity/Q1211063',), ('http://www.wikidata.org/entity/Q1251918',)], 'repaired': True, 'error': None}

[SPARQL Query Used]
SELECT ?film
WHERE {
  ?film wdt:P161 ?castMember .
}
LIMIT 20

[Repaired?]
True

[Results]
film
http://www.wikidata.org/entity/Q100104941
http://www.wikidata.org/entity/Q10331225
http://www.wikidata.org/entity/Q1068968
http://www.wikidata.org/entity/Q1070468
```

Figure 3: Retrieved results after SPARQL execution

The results show valid Wikidata entities corresponding to films, demonstrating the effectiveness of the retrieval-based approach.

This example highlights the clear improvement of RAG over the baseline model, both in terms of accuracy and interpretability.

4.6 Evaluation Results

Question	Baseline Answer	SPARQL Result	Correct
What entities are linked to a country?	Vague explanation	Retrieves entities using `wdt:P17`	Yes
What is the type of entities?	Generic answer	Correct use of `wdt:P31`	Partial
Which entities have an occupation?	Imprecise	Uses `wdt:P106`	No
Where were entities born?	Approximate answer	Uses `wdt:P19`	Yes
General entity query	Hallucinated output	Structured RDF results	Yes

Table 2: Comparison between baseline and SPARQL-based answers

4.7 Analysis

The results highlight a clear difference between the baseline model and the RAG system.

First, the baseline model produces vague, generic, and sometimes incorrect answers. It is not grounded in the knowledge graph and cannot reliably answer domain-specific queries.

In contrast, the RAG system generates structured and factual answers by querying the RDF graph. This significantly improves reliability and interpretability.

However, several limitations were observed. SPARQL generation may fail due to incorrect predicates, syntax errors, or missing constraints. In such cases, the self-repair mechanism plays a key role by correcting invalid queries using error feedback.

Additionally, the quality of results depends on the schema summary. If the summary is too large, it increases latency and may confuse the model. If it is too small, important information may be missing.

4.8 Conclusion

This experiment demonstrates the effectiveness of combining language models with structured knowledge graphs. The RAG approach significantly outperforms a standard LLM by grounding answers in factual data.

The self-repair mechanism improves robustness and allows the system to recover from query generation errors.

Future improvements could focus on better schema summarization, more advanced prompting strategies, and scaling the approach to larger knowledge graphs.

5 Critical Reflection

5.1 Impact of KB Quality

The quality of the knowledge base has a direct impact on all downstream tasks. Noisy or weakly connected entities negatively affect both reasoning and embedding performance.

5.2 Noise Issues

The expansion process introduced noise, especially through loosely related entities. This reduces the coherence of the graph and impacts model performance.

5.3 Rule-based vs Embedding-based Reasoning

Rule-based reasoning (SWRL) is precise and interpretable but limited in scalability.

Embedding-based reasoning is more flexible and scalable, but less interpretable and sensitive to data quality.

5.4 Future Improvements

Future work should focus on:

- better entity filtering
- improved alignment strategies
- more advanced embedding models

- hybrid reasoning combining rules and embeddings